

HLLC-2D Riemann求解器

技术文档与C++程序框架

基于论文：

Part I: A robust and contact resolving Riemann solver on unstructured mesh,
Euler method (JCP 268, 2014, 432-455)

Part II: A robust and contact resolving Riemann solver on unstructured mesh,
ALE method (JCP 268, 2014, 456-484)

作者：Zhijun Shen, Wei Yan, Guangwei Yuan

技术文档生成日期：2025年

目录

第1章 引言

- 1.1 背景介绍
- 1.2 问题陈述
- 1.3 文档结构

第2章 控制方程

- 2.1 Euler方程的守恒形式
- 2.2 状态方程
- 2.3 特征分析

第3章 HLLC-2D求解器数学理论

- 3.1 传统HLLC求解器回顾
- 3.2 HLLC-2D的核心创新
- 3.3 波速估计
- 3.4 Star区域状态计算

第4章 Nodal Solver详细推导

- 4.1 守恒条件的推导
- 4.2 M矩阵的构造
- 4.3 节点速度求解算法
- 4.4 与Lagrangian框架的联系

第5章 ALE框架扩展

- 5.1 ALE控制方程

5.2 网格运动策略

5.3 时间步长限制

第6章 C++程序框架

6.1 整体架构

6.2 核心类设计

6.3 Nodal Solver实现详解

6.4 完整代码框架

第7章 数值实验简述

第8章 总结

第1章 引言

1.1 背景介绍

计算流体力学（Computational Fluid Dynamics, CFD）是研究流体运动规律的重要数值方法。在可压缩流体模拟中，Riemann求解器是有限体积方法的核心组件，它负责计算网格界面上的数值通量。

传统的HLL（Harten-Lax-van Leer）求解器是一种近似Riemann求解器，它假设Riemann问题的解由两个波（最左和最右波）分隔三个常数状态区域组成。HLLC（HLL-Contact）求解器在此基础上增加了对接触间断的解析，能够更准确地捕捉接触波和剪切波。

然而，传统的HLLC求解器是为**一维**Riemann问题设计的，它假设接触波两侧的压力相等。在**二维非结构网格**上，这种假设会导致节点处通量不一致的问题，影响计算的稳定性和精度。

核心问题

在非结构网格上，多条边在一个节点交汇。如果每条边独立求解一维Riemann问题，节点处的通量可能不满足守恒条件，导致数值不稳定。

1.2 问题陈述

Shen、Yan和Yuan在2014年发表的系列论文提出了**HLLC-2D求解器**，其核心创新包括：

- 放松压力相等条件**：允许接触波两侧的压力不同
- 引入节点求解器（Nodal Solver）**：计算节点接触速度，确保所有交汇于该节点的边通量一致
- 统一Euler和ALE框架**：通过选择不同的网格运动速度，可以退化为纯Eulerian或纯Lagrangian方法

1.3 文档结构

本文档按照以下结构组织：

- 第2章介绍Euler方程的数学形式
- 第3章详细阐述HLLC-2D求解器的数学理论
- 第4章是本文档的核心，详细推导Nodal Solver
- 第5章介绍ALE框架的扩展
- 第6章提供完整的C++程序框架
- 第7章简述数值实验
- 第8章总结

第2章 控制方程

2.1 Euler方程的守恒形式

二维可压缩Euler方程的守恒形式为：

$$\frac{\partial \mathbf{U}}{\partial t} + \frac{\partial \mathbf{F}}{\partial x} + \frac{\partial \mathbf{G}}{\partial y} = 0$$

其中，守恒变量 $\mathbf{U}$ 和通量 $\mathbf{F}$ 、 $\mathbf{G}$ 定义为：

$$\mathbf{U} = \begin{pmatrix} \rho \\ \rho u \\ \rho v \\ \rho E \end{pmatrix}, \quad \mathbf{F} = \begin{pmatrix} \rho u \\ \rho u^2 + p \\ \rho uv \\ (\rho E + p)u \end{pmatrix}, \quad \mathbf{G} = \begin{pmatrix} \rho v \\ \rho uv \\ \rho v^2 + p \\ (\rho E + p)v \end{pmatrix}$$

各变量的物理意义如下：

变量	符号	物理意义
密度	ρ	单位体积的质量
x方向速度	u	流体在x方向的速度分量
y方向速度	v	流体在y方向的速度分量
总能量	E	单位质量的总能量
压力	p	流体静压

2.2 状态方程

对于理想气体，状态方程为：

$$p = (\gamma - 1)\rho e = (\gamma - 1) \left(\rho E - \frac{1}{2}\rho(u^2 + v^2) \right)$$

其中 γ 是比热比（对于空气，通常取 $\gamma = 1.4$ ）， e 是比内能。

声速 a 的计算公式为：

$$a = \sqrt{\frac{\gamma p}{\rho}}$$

2.3 特征分析

Euler方程的特征值（在一维情况下）为：

$$\lambda_1 = u - a, \quad \lambda_2 = \lambda_3 = u, \quad \lambda_4 = u + a$$

对应的波系包括：

- $\lambda_1 = u - a$ ：向左传播的声波（稀疏波或激波）
- $\lambda_2 = \lambda_3 = u$ ：接触间断（密度和切向速度可能不连续）
- $\lambda_4 = u + a$ ：向右传播的声波（稀疏波或激波）

在Riemann问题中，初始间断会分解为上述波系，中间区域被接触波分隔为两个状态（ L^* 和 R^* ）。

第3章 HLLC-2D求解器数学理论

3.1 传统HLLC求解器回顾

传统的HLLC求解器假设一维Riemann问题的解包含四个状态区域，由三个波分隔：

$$\mathbf{U}(x, t) = \begin{cases} \mathbf{U}_L & \text{if } x/t \leq S_L \\ \mathbf{U}_{L*} & \text{if } S_L < x/t \leq S_* \\ \mathbf{U}_{R*} & \text{if } S_* < x/t \leq S_R \\ \mathbf{U}_R & \text{if } x/t > S_R \end{cases}$$

其中， S_L 和 S_R 是最左和最右波速， S_* 是接触波速度。

传统HLLC的关键假设

传统HLLC求解器假设接触波两侧的压力相等：

$$p_{L*} = p_{R*} = p_*$$

基于质量、动量和能量守恒，可以推导出Star区域的状态：

$$\rho_{K*} = \rho_K \frac{S_K - u_K}{S_K - S_*}, \quad K = L, R$$

$$u_* = S_*, \quad p_* = p_K + \rho_K (S_K - u_K)(S_* - u_K)$$

$$\mathbf{U}_{K*} = \frac{(S_K - u_K)\mathbf{U}_K + (p_K - p_*)\mathbf{D}_K}{S_K - S_*}$$

其中 $\mathbf{D}_K = (0, 1, 0, S_*)^T$ 。

3.2 HLLC-2D的核心创新

HLLC-2D求解器的核心创新在于**放松压力相等条件**，允许接触波两侧的压力不同。这使得求解器能够更好地处理二维非结构网格上的复杂流动。

HLLC-2D的关键改进

在HLLC-2D中，接触波两侧的压力可以不同：

$$p_{L*} \neq p_{R*}$$

这一改变使得求解器能够：

1. 更好地处理角点（节点）处的多波交汇问题
2. 确保所有交汇于同一节点的边通量满足守恒条件
3. 提高接触波和剪切波的捕捉精度

HLLC-2D的通量公式（对于每条边）为：

$$\mathbf{F}_{f,q}^c = \begin{cases} \mathbf{U}_L(S_* - w) + \mathbf{D}_{L*} & \text{if } S_L < w \leq S_* \\ \mathbf{U}_R(S_* - w) + \mathbf{D}_{R*} & \text{if } S_* < w \leq S_R \end{cases}$$

其中：

- w 是网格运动速度（在ALE框架中）

- $S_* = \mathbf{u}_*^q \cdot \mathbf{N}_f^c$ 是接触波速度
- $\mathbf{D}_{K*} = (0, p_{K*}N_x, p_{K*}N_y, p_{K*}S_*)^T$
- $p_{K*} = p_K + \rho_K(S_K - u_K)(S_* - u_K)$

3.3 波速估计

波速估计是Riemann求解器的关键部分。HLLC-2D采用以下波速估计公式：

3.3.1 Davis估计

$$S_L = \min(u_L - a_L, u_R - a_R), \quad S_R = \max(u_L + a_L, u_R + a_R)$$

3.3.2 Einfeldt估计（推荐）

$$S_L = \tilde{u} - \tilde{a}, \quad S_R = \tilde{u} + \tilde{a}$$

其中Roe平均量为：

$$\tilde{\rho} = \sqrt{\rho_L \rho_R}, \quad \tilde{u} = \frac{\sqrt{\rho_L} u_L + \sqrt{\rho_R} u_R}{\sqrt{\rho_L} + \sqrt{\rho_R}}$$

$$\tilde{H} = \frac{\sqrt{\rho_L} H_L + \sqrt{\rho_R} H_R}{\sqrt{\rho_L} + \sqrt{\rho_R}}, \quad \tilde{a} = \sqrt{(\gamma - 1) \left(\tilde{H} - \frac{\tilde{u}^2}{2} \right)}$$

其中 $H = E + p/\rho$ 是比总焓。

3.3.3 接触波速度估计

一维HLLC接触波速度可以通过压力-速度一致性条件估计：

$$S_* = \frac{p_R - p_L + \rho_L u_L (S_L - u_L) - \rho_R u_R (S_R - u_R)}{\rho_L (S_L - u_L) - \rho_R (S_R - u_R)}$$

3.4 Star区域状态计算

在HLLC-2D中，Star区域的状态计算需要考虑二维效应。对于每个状态 $K \in \{L, R\}$ ：

3.4.1 密度

$$\rho_{K*} = \rho_K \frac{S_K - u_K}{S_K - S_*}$$

3.4.2 速度

法向速度等于接触波速度：

$$u_{K*,n} = S_*$$

切向速度保持不变（接触间断特性）：

$$u_{K*,\tau} = u_{K,\tau}$$

3.4.3 压力

$$p_{K*} = p_K + \rho_K (S_K - u_K) (S_* - u_K)$$

3.4.4 总能量

$$E_{K*} = \frac{(S_K - u_K)E_K - p_K u_K + p_{K*} S_*}{S_K - S_*}$$

重要说明

在HLLC-2D中， p_{L*} 和 p_{R*} 通常不相等。这一特性使得求解器能够更好地处理节点处的多波交汇问题，但需要通过Nodal Solver来保证守恒性。

第4章 Nodal Solver详细推导

Nodal Solver是HLLC-2D求解器的核心创新，它负责计算节点接触速度 $\mathbf{u}_*^q$ ，确保所有交汇于该节点的边通量满足守恒条件。

4.1 守恒条件的推导

考虑节点 q 及其相邻的 N 条边。每条边 k 连接节点 q 和相邻节点，具有单位外法向 $\mathbf{N}_k$ 和长度 L_k 。

守恒条件的物理基础

在节点 q 处，所有交汇边的通量之和应该为零（封闭区域的质量、动量和能量守恒）：

$$\sum_{k=1}^N L_k \mathbf{F}_k^c \cdot \mathbf{N}_k = 0$$

其中 $\mathbf{F}_k^c$ 是边 k 的半通量（half-edge flux）。

对于每条边 k ，定义左右状态（相对于边的法向）和对应的波速：

- 左状态： $\mathbf{U}_{L,k}$ ，波速 $S_{L,k}$
- 右状态： $\mathbf{U}_{R,k}$ ，波速 $S_{R,k}$
- 一维HLLC接触速度： $\nu_{*,k}$

定义系数 α ：

$$\alpha_{L,k} = -\rho_{L,k}(S_{L,k} - u_{L,k})$$

$$\alpha_{R,k} = \rho_{R,k}(S_{R,k} - u_{R,k})$$

半通量的表达式为：

$$\mathbf{F}_k^c = \begin{cases} \mathbf{U}_{L,k}(\mathbf{u}_*^q \cdot \mathbf{N}_k - w_k) + \mathbf{D}_{L*,k} & (\text{左状态活跃}) \\ \mathbf{U}_{R,k}(\mathbf{u}_*^q \cdot \mathbf{N}_k - w_k) + \mathbf{D}_{R*,k} & (\text{右状态活跃}) \end{cases}$$

4.2 M矩阵的构造

将守恒条件展开，可以得到关于节点速度 $\mathbf{u}_*^q$ 的线性方程组。首先定义：

$$\beta_{L,k} = \alpha_{L,k}(\nu_{*,k} - w_k) + p_{L,k}$$

$$\beta_{R,k} = \alpha_{R,k}(\nu_{*,k} - w_k) + p_{R,k}$$

守恒条件可以写成：

$$\sum_{k=1}^N L_k [(\alpha_{L,k} + \alpha_{R,k})(\mathbf{u}_*^q \cdot \mathbf{N}_k) \mathbf{N}_k + \beta_k \mathbf{N}_k] = 0$$

整理后得到矩阵形式：

$$\mathbf{M} \mathbf{u}_*^q = \mathbf{b}$$

M矩阵的详细构造

M矩阵是一个 2×2 矩阵，定义为：

$$\mathbf{M} = \sum_{k=1}^N L_k (\alpha_{L,k} + \alpha_{R,k}) \mathbf{N}_k \otimes \mathbf{N}_k$$

展开分量形式（设 $\mathbf{N}_k = (N_{k,x}, N_{k,y})$ ）：

$$\mathbf{M} = \begin{pmatrix} \sum_k L_k (\alpha_{L,k} + \alpha_{R,k}) N_{k,x}^2 & \sum_k L_k (\alpha_{L,k} + \alpha_{R,k}) N_{k,x} N_{k,y} \\ \sum_k L_k (\alpha_{L,k} + \alpha_{R,k}) N_{k,x} N_{k,y} & \sum_k L_k (\alpha_{L,k} + \alpha_{R,k}) N_{k,y}^2 \end{pmatrix}$$

右端项b的构造

$$\mathbf{b} = - \sum_{k=1}^N L_k [(\alpha_{L,k} + \alpha_{R,k}) \nu_{*,k} \mathbf{N}_k + (p_{L*,k} - p_{R*,k}) \mathbf{N}_k]$$

或者更简洁地：

$$\mathbf{b} = \sum_{k=1}^N L_k (\alpha_{L,k} + \alpha_{R,k}) \nu_{*,k} \mathbf{N}_k$$

（假设压力项的贡献在求和时相互抵消或已包含在简化形式中）

4.3 节点速度求解算法

算法：Nodal Solver计算节点接触速度

1. 输入：节点 q ，相邻边集合 $\{k\}$ ，每条边的左右状态
2. 初始化： $\mathbf{M} = \mathbf{0}$ ， $\mathbf{b} = \mathbf{0}$
3. 循环遍历每条边 k ：
 - 计算波速 $S_{L,k}$ ， $S_{R,k}$
 - 计算一维HLLC接触速度 $\nu_{*,k}$
 - 计算系数 $\alpha_{L,k} = -\rho_{L,k}(S_{L,k} - u_{L,k})$
 - 计算系数 $\alpha_{R,k} = \rho_{R,k}(S_{R,k} - u_{R,k})$
 - 累加M矩阵： $\mathbf{M} += L_k(\alpha_{L,k} + \alpha_{R,k})\mathbf{N}_k\mathbf{N}_k^T$
 - 累加右端项： $\mathbf{b} += L_k(\alpha_{L,k} + \alpha_{R,k})\nu_{*,k}\mathbf{N}_k$
4. 求解线性方程组： $\mathbf{u}_*^q = \mathbf{M}^{-1}\mathbf{b}$
5. 输出：节点接触速度 $\mathbf{u}_*^q$

数值稳定性考虑

在实际计算中，需要注意：

1. 当 $\alpha_{L,k} + \alpha_{R,k} \approx 0$ 时，需要添加正则化
2. M矩阵必须是正定的，否则需要特殊处理
3. 对于边界节点，需要根据边界条件修改算法

4.4 与Lagrangian框架的联系

HLLC-2D的一个重要特性是它可以自然地退化为Lagrangian方法。当网格运动速度等于节点接触速度时：

$$\mathbf{w}_q = \mathbf{u}_*^q$$

这对应于纯Lagrangian方法，网格随流体一起运动。

当网格固定时：

$$\mathbf{w}_q = \mathbf{0}$$

这退化为纯Eulerian方法。

在ALE（Arbitrary Lagrangian-Eulerian）框架中，网格运动速度可以是任意值，提供了更大的灵活性：

$$\mathbf{w}_q = \kappa \mathbf{u}_*^q + (1 - \kappa) \mathbf{w}_{\text{other}}$$

其中 $\kappa \in [0, 1]$ 是混合系数， $\mathbf{w}_{\text{other}}$ 可以是零（Eulerian）或其他网格运动策略。

第5章 ALE框架扩展

5.1 ALE控制方程

ALE (Arbitrary Lagrangian-Eulerian) 方法是一种结合了Eulerian和Lagrangian优势的框架。在ALE框架下，控制方程需要在移动网格上重新推导。

ALE形式的Euler方程

对于随网格运动的控制体，守恒方程为：

$$\frac{d}{dt} \int_{V(t)} \mathbf{U} dV + \oint_{\partial V(t)} (\mathbf{F} - \mathbf{U} \otimes \mathbf{w}) \cdot \mathbf{n} dS = 0$$

其中 $\mathbf{w}$ 是网格运动速度。

半离散形式为：

$$\frac{d(\mathbf{U}_i V_i)}{dt} = - \sum_{f \in \partial V_i} \mathbf{F}_f^c \cdot \mathbf{N}_f L_f$$

其中 $\mathbf{F}_f^c$ 是ALE数值通量：

$$\mathbf{F}_f^c = \mathbf{F}(\mathbf{U}) - \mathbf{U} \otimes \mathbf{w}$$

5.2 网格运动策略

在ALE框架中，网格运动策略的选择至关重要。HLLC-2D提供了灵活的网格运动选项：

5.2.1 Lagrangian模式

$$\mathbf{w}_q = \mathbf{u}_*^q$$

网格随流体一起运动，适用于追踪物质界面，但可能导致网格畸变。

5.2.2 Eulerian模式

$$\mathbf{w}_q = \mathbf{0}$$

网格固定不动，适用于大变形问题，但界面捕捉精度较低。

5.2.3 混合模式

$$\mathbf{w}_q = \kappa \mathbf{u}_*^q + (1 - \kappa) \mathbf{w}_{\text{smooth}}$$

其中 κ 是混合系数， $\mathbf{w}_{\text{smooth}}$ 是通过平滑或优化得到的网格速度。

5.2.4 网格重分策略

当网格质量下降到一定阈值时，需要进行网格重分（remeshing）：

1. 计算网格质量指标（如Jacobian行列式、长宽比等）
2. 如果质量指标低于阈值，触发重分
3. 生成新网格，将旧解插值到新网格
4. 继续计算

5.3 时间步长限制

ALE框架下的时间步长限制比纯Eulerian更复杂，需要考虑：

5.3.1 CFL条件

$$\Delta t_{\text{CFL}} = C_{\text{CFL}} \cdot \min_i \left(\frac{V_i}{\sum_f (|u_{f,n} - w_{f,n}| + a_f) A_f} \right)$$

其中 C_{CFL} 是CFL数（通常取0.25-0.5）， a_f 是声速。

5.3.2 体积变化限制

为了防止网格体积变化过快，需要添加体积变化限制：

$$\Delta t_{\text{vol}} = C_{\text{vol}} \cdot \min_i \left(\frac{V_i}{|dV_i/dt|} \right)$$

其中 C_{vol} 是体积变化限制系数（通常取0.1）。

5.3.3 综合时间步长

$$\Delta t = \min(\Delta t_{\text{CFL}}, \Delta t_{\text{vol}})$$

实践建议

在实际计算中，建议：

- 初始阶段使用较小的CFL数
- 根据网格质量动态调整时间步长
- 对于强激波问题，使用更严格的时间步长限制

第6章 C++程序框架

6.1 整体架构

HLLC-2D求解器的C++程序采用面向对象设计，主要包含以下模块：

- 基础模块：向量、矩阵等数学工具
- 网格模块：非结构网格的数据结构和管理
- 物理模块：状态变量、状态方程
- 求解器模块：Riemann求解器、Nodal Solver
- 时间积分模块：显式/隐式时间推进
- IO模块：输入输出、可视化

6.2 核心类设计

6.2.1 Vector2D - 二维向量类

```
class Vector2D {
public:
    double x, y;

    // 构造函数
    Vector2D(double x = 0.0, double y = 0.0) : x(x), y(y) {}

    // 向量运算
    Vector2D operator+(const Vector2D& other) const {
        return Vector2D(x + other.x, y + other.y);
    }

    Vector2D operator-(const Vector2D& other) const {
        return Vector2D(x - other.x, y - other.y);
    }

    Vector2D operator*(double scalar) const {
        return Vector2D(x * scalar, y * scalar);
    }

    // 点积
    double dot(const Vector2D& other) const {
```

```

        return x * other.x + y * other.y;
    }

    // 叉积 (二维标量)
    double cross(const Vector2D& other) const {
        return x * other.y - y * other.x;
    }

    // 模长
    double magnitude() const {
        return std::sqrt(x * x + y * y);
    }

    // 单位化
    Vector2D normalized() const {
        double mag = magnitude();
        if (mag > 1e-15) {
            return Vector2D(x / mag, y / mag);
        }
        return Vector2D(0.0, 0.0);
    }

    // 外积 (产生2x2矩阵)
    void outerProduct(double result[2][2]) const {
        result[0][0] = x * x; result[0][1] = x * y;
        result[1][0] = y * x; result[1][1] = y * y;
    }
};

```

6.2.2 State - 守恒变量状态

```

class State {
public:
    double rho;           // 密度
    double rhoU;          // x方向动量
    double rhoV;          // y方向动量
    double rhoE;          // 总能量

    static constexpr double GAMMA = 1.4;

    // 构造函数
    State(double rho = 0.0, double rhoU = 0.0,
          double rhoV = 0.0, double rhoE = 0.0)
        : rho(rho), rhoU(rhoU), rhoV(rhoV), rhoE(rhoE) {}

    // 速度分量
    double u() const { return rhoU / rho; }
    double v() const { return rhoV / rho; }

    // 速度向量
    Vector2D velocity() const {

```

```

        return Vector2D(u(), v());
    }

    // 动能
    double kineticEnergy() const {
        return 0.5 * (rhoU * rhoU + rhoV * rhoV) / rho;
    }

    // 内能
    double internalEnergy() const {
        return rhoE / rho - kineticEnergy() / rho;
    }

    // 压力 (状态方程)
    double pressure() const {
        return (GAMMA - 1.0) * (rhoE - kineticEnergy());
    }

    // 声速
    double soundSpeed() const {
        return std::sqrt(GAMMA * pressure() / rho);
    }

    // 总焓
    double enthalpy() const {
        return (rhoE + pressure()) / rho;
    }

    // 法向速度
    double normalVelocity(const Vector2D& normal) const {
        return u() * normal.x + v() * normal.y;
    }

    // 切向速度
    double tangentialVelocity(const Vector2D& normal) const {
        Vector2D tangent(-normal.y, normal.x);
        return u() * tangent.x + v() * tangent.y;
    }
};

```

6.2.3 Matrix2x2 - 2×2 矩阵类

```

class Matrix2x2 {
public:
    double data[2][2];

    Matrix2x2() {
        data[0][0] = data[0][1] = data[1][0] = data[1][1] = 0.0;
    }

    // 矩阵-向量乘法

```

```

Vector2D operator*(const Vector2D& v) const {
    return Vector2D(
        data[0][0] * v.x + data[0][1] * v.y,
        data[1][0] * v.x + data[1][1] * v.y
    );
}

// 矩阵加法
Matrix2x2 operator+(const Matrix2x2& other) const {
    Matrix2x2 result;
    for (int i = 0; i < 2; ++i)
        for (int j = 0; j < 2; ++j)
            result.data[i][j] = data[i][j] + other.data[i][j];
    return result;
}

Matrix2x2& operator+=(const Matrix2x2& other) {
    for (int i = 0; i < 2; ++i)
        for (int j = 0; j < 2; ++j)
            data[i][j] += other.data[i][j];
    return *this;
}

// 标量乘法
Matrix2x2 operator*(double scalar) const {
    Matrix2x2 result;
    for (int i = 0; i < 2; ++i)
        for (int j = 0; j < 2; ++j)
            result.data[i][j] = data[i][j] * scalar;
    return result;
}

// 行列式
double determinant() const {
    return data[0][0] * data[1][1] - data[0][1] * data[1][0];
}

// 求逆矩阵
bool inverse(Matrix2x2& result) const {
    double det = determinant();
    if (std::abs(det) < 1e-15) {
        return false; // 矩阵奇异
    }
    double invDet = 1.0 / det;
    result.data[0][0] = data[1][1] * invDet;
    result.data[0][1] = -data[0][1] * invDet;
    result.data[1][0] = -data[1][0] * invDet;
    result.data[1][1] = data[0][0] * invDet;
    return true;
}

// 解线性方程组  $M * x = b$ 
bool solve(const Vector2D& b, Vector2D& x) const {
    Matrix2x2 inv;

```



```

        if (!inverse(inv)) {
            return false;
        }
        x = inv * b;
        return true;
    }
};

```

6.3 Nodal Solver实现详解

6.3.1 数据结构定义

```

// 边结构
struct Edge {
    int leftCell;           // 左侧单元索引
    int rightCell;          // 右侧单元索引
    int vertex1;            // 端点1
    int vertex2;            // 端点2
    Vector2D normal;        // 单位外法向 (从left指向right)
    double length;          // 边长
    Vector2D midpoint;      // 中点坐标
};

// 顶点/节点结构
struct Vertex {
    Vector2D position;      // 位置
    Vector2D velocity;      // 节点速度 (ALE框架)
    std::vector<int> edges;  // 相邻边索引
    std::vector<int> cells;  // 相邻单元索引
};

// 单元结构
struct Cell {
    std::vector<int> vertices; // 顶点索引
    std::vector<int> edges;    // 边索引
    State state;              // 守恒变量
    double volume;            // 体积/面积
    Vector2D centroid;        // 质心
};

```

6.3.2 NodalSolver类

```

class NodalSolver {
public:
    NodalSolver(double gamma = 1.4) : gamma_(gamma) {}

    // 计算节点接触速度 (核心函数)

```

```

Vector2D computeContactVelocity(
    const Vertex& vertex,
    const std::vector<Edge>& edges,
    const std::vector<Cell>& cells,
    const Vector2D& meshVelocity = Vector2D(0.0, 0.0)
) {
    // 初始化M矩阵和右端项
    Matrix2x2 M;
    Vector2D b(0.0, 0.0);

    // 遍历所有相邻边
    for (int edgeIdx : vertex.edges) {
        const Edge& edge = edges[edgeIdx];

        // 确定左右状态（相对于当前节点）
        bool isLeft = (edge.vertex1 == &vertex - &vertices[0]) ||
            (edge.vertex2 == &vertex - &vertices[0]);

        const State& stateL = cells[edge.leftCell].state;
        const State& stateR = cells[edge.rightCell].state;

        // 计算波速
        double SL, SR;
        computeWaveSpeeds(stateL, stateR, edge.normal, SL, SR);

        // 计算一维HLLC接触速度
        double nuStar = computeHLLCContactSpeed(
            stateL, stateR, edge.normal, SL, SR
        );

        // 计算alpha系数
        double uL = stateL.normalVelocity(edge.normal);
        double uR = stateR.normalVelocity(edge.normal);

        double alphaL = -stateL.rho * (SL - uL);
        double alphaR = stateR.rho * (SR - uR);
        double alphaSum = alphaL + alphaR;

        // 正则化处理
        if (std::abs(alphaSum) < 1e-15) {
            alphaSum = 1e-15;
        }

        // 累加M矩阵:  $M += L_k * (\alpha_L + \alpha_R) * N_k * N_k^T$ 
        double factor = edge.length * alphaSum;
        Matrix2x2 outer;
        edge.normal.outerProduct(outer.data);
        M += outer * factor;

        // 累加右端项:  $b += L_k * (\alpha_L + \alpha_R) * nu^*_k * N_k$ 
        double bFactor = edge.length * alphaSum * nuStar;
        b = b + edge.normal * bFactor;
    }
}

```

```

        // 求解线性方程组  $M * u^* = b$ 
        Vector2D uStar;
        if (!M.solve(b, uStar)) {
            // 矩阵奇异, 使用备选方案
            uStar = computeFallbackVelocity(vertex, edges, cells);
        }

        return uStar;
    }

private:
    double gamma_;
    std::vector<Vertex> vertices; // 用于索引计算

    // 计算波速 (Einfeldt估计)
    void computeWaveSpeeds(
        const State& left,
        const State& right,
        const Vector2D& normal,
        double& SL,
        double& SR
    ) {
        double uL = left.normalVelocity(normal);
        double uR = right.normalVelocity(normal);
        double aL = left.soundSpeed();
        double aR = right.soundSpeed();

        // Roe平均
        double sqrtRhoL = std::sqrt(left.rho);
        double sqrtRhoR = std::sqrt(right.rho);
        double invSum = 1.0 / (sqrtRhoL + sqrtRhoR);

        double uTilde = (sqrtRhoL * uL + sqrtRhoR * uR) * invSum;
        double HTilde = (sqrtRhoL * left.enthalpy() +
                        sqrtRhoR * right.enthalpy()) * invSum;
        double aTilde = std::sqrt((gamma_ - 1.0) *
                                   (HTilde - 0.5 * uTilde * uTilde));

        SL = std::min(uL - aL, uTilde - aTilde);
        SR = std::max(uR + aR, uTilde + aTilde);
    }

    // 计算一维HLLC接触速度
    double computeHLLCContactSpeed(
        const State& left,
        const State& right,
        const Vector2D& normal,
        double SL,
        double SR
    ) {
        double uL = left.normalVelocity(normal);
        double uR = right.normalVelocity(normal);
        double pL = left.pressure();
        double pR = right.pressure();
    }

```

```

        double numerator = pR - pL + left.rho * uL * (SL - uL)
                        - right.rho * uR * (SR - uR);
        double denominator = left.rho * (SL - uL)
                        - right.rho * (SR - uR);

        if (std::abs(denominator) < 1e-15) {
            return 0.5 * (uL + uR);
        }

        return numerator / denominator;
    }

    // 备选速度计算（当M矩阵奇异时）
    Vector2D computeFallbackVelocity(
        const Vertex& vertex,
        const std::vector<Edge>& edges,
        const std::vector<Cell>& cells
    ) {
        Vector2D avgVelocity(0.0, 0.0);
        double totalWeight = 0.0;

        for (int edgeIdx : vertex.edges) {
            const Edge& edge = edges[edgeIdx];
            double weight = edge.length;

            // 使用相邻单元速度的加权平均
            Vector2D vL = cells[edge.leftCell].state.velocity();
            Vector2D vR = cells[edge.rightCell].state.velocity();

            avgVelocity = avgVelocity + (vL + vR) * (0.5 * weight);
            totalWeight += weight;
        }

        if (totalWeight > 1e-15) {
            return avgVelocity * (1.0 / totalWeight);
        }

        return Vector2D(0.0, 0.0);
    }
};

```

6.3.3 HLLC2DSolver类

```

class HLLC2DSolver {
public:
    HLLC2DSolver(double gamma = 1.4)
        : gamma_(gamma), nodalSolver_(gamma) {}

    // 计算边的数值通量
    State computeFlux(

```

```

    const State& left,
    const State& right,
    const Vector2D& normal,
    double edgeLength,
    const Vector2D& nodalVelocity,
    const Vector2D& meshVelocity
) {
    // 计算法向速度
    double uL = left.normalVelocity(normal);
    double uR = right.normalVelocity(normal);

    // 计算波速
    double SL, SR;
    computeWaveSpeeds(left, right, normal, SL, SR);

    // 计算接触波速度 (使用节点速度)
    double SStar = nodalVelocity.dot(normal);

    // 网格运动速度在法向的投影
    double w = meshVelocity.dot(normal);

    // 计算Star区域的压力
    double pLstar = left.pressure() + left.rho * (SL - uL) * (SStar - SL);
    double pRstar = right.pressure() + right.rho * (SR - uR) * (SStar - SR);

    // 根据波的位置选择通量
    if (SStar <= w) {
        // 左Star区域活跃
        return computeStarFlux(left, normal, SL, SStar, w, pLstar);
    } else {
        // 右Star区域活跃
        return computeStarFlux(right, normal, SR, SStar, w, pRstar);
    }
}

// 更新所有节点的速度
void updateNodalVelocities(
    std::vector<Vertex>& vertices,
    const std::vector<Edge>& edges,
    const std::vector<Cell>& cells
) {
    for (auto& vertex : vertices) {
        vertex.velocity = nodalSolver_.computeContactVelocity(
            vertex, edges, cells
        );
    }
}

private:
    double gamma_;
    NodalSolver nodalSolver_;

    void computeWaveSpeeds(
        const State& left,

```

```

    const State& right,
    const Vector2D& normal,
    double& SL,
    double& SR
) {
    // 与NodalSolver中相同的实现
    double uL = left.normalVelocity(normal);
    double uR = right.normalVelocity(normal);
    double aL = left.soundSpeed();
    double aR = right.soundSpeed();

    double sqrtRhoL = std::sqrt(left.rho);
    double sqrtRhoR = std::sqrt(right.rho);
    double invSum = 1.0 / (sqrtRhoL + sqrtRhoR);

    double uTilde = (sqrtRhoL * uL + sqrtRhoR * uR) * invSum;
    double HTilde = (sqrtRhoL * left.enthalpy() +
                    sqrtRhoR * right.enthalpy()) * invSum;
    double aTilde = std::sqrt((gamma_ - 1.0) *
                              (HTilde - 0.5 * uTilde * uTilde));

    SL = std::min(uL - aL, uTilde - aTilde);
    SR = std::max(uR + aR, uTilde + aTilde);
}

State computeStarFlux(
    const State& state,
    const Vector2D& normal,
    double SK,
    double SStar,
    double w,
    double pStar
) {
    double u = state.normalVelocity(normal);
    double factor = (SK - u) / (SK - SStar);

    // Star区域密度
    double rhoStar = state.rho * factor;

    // Star区域动量
    Vector2D vel = state.velocity();
    Vector2D velStar = vel + normal * (SStar - u);

    double rhoUstar = rhoStar * velStar.x;
    double rhoVstar = rhoStar * velStar.y;

    // Star区域能量
    double EStar = ((SK - u) * state.rhoE - state.pressure() * u
                  + pStar * SStar) / (SK - SStar);
    double rhoEstar = rhoStar * EStar;

    // 构造Star区域状态
    State stateStar(rhoStar, rhoUstar, rhoVstar, rhoEstar);

```

```

// 计算通量:  $F = U(S^* - w) + D^*$ 
double fluxFactor = SStar - w;

double fluxRho = rhoStar * fluxFactor;
double fluxRhoU = rhoUstar * fluxFactor + pStar * normal.x;
double fluxRhoV = rhoVstar * fluxFactor + pStar * normal.y;
double fluxRhoE = rhoEstar * fluxFactor + pStar * SStar;

return State(fluxRho, fluxRhoU, fluxRhoV, fluxRhoE);
}
};

```

6.4 完整代码框架

6.4.1 Mesh类

```

class Mesh {
public:
    std::vector<Vertex> vertices;
    std::vector<Edge> edges;
    std::vector<Cell> cells;

    // 从文件读取网格
    bool loadFromFile(const std::string& filename);

    // 计算几何量
    void computeGeometry();

    // 计算单元体积/面积
    double computeCellVolume(int cellIdx);

    // 计算单元质心
    Vector2D computeCellCentroid(int cellIdx);

    // 计算边的法向
    void computeEdgeNormals();

    // 更新网格 (ALE框架)
    void updatePositions(double dt);

    // 网格质量检查
    double computeMeshQuality();

    // 获取单元邻居
    std::vector<int> getCellNeighbors(int cellIdx);

    // 获取顶点相邻单元
    std::vector<int> getVertexCells(int vertexIdx);
};

```

6.4.2 TimeIntegrator类

```
class TimeIntegrator {
public:
    enum Scheme {
        FORWARD_EULER,
        RK2,
        RK3_SSP
    };

    TimeIntegrator(Scheme scheme = RK2, double cfl = 0.5)
        : scheme_(scheme), cfl_(cfl) {}

    // 单步推进
    void step(
        Mesh& mesh,
        HLLC2DSolver& solver,
        double dt
    );

    // 计算时间步长
    double computeTimeStep(const Mesh& mesh);

    // 计算CFL时间步长
    double computeCFLTimeStep(const Mesh& mesh);

    // 计算体积变化时间步长 (ALE)
    double computeVolumeTimeStep(const Mesh& mesh);

private:
    Scheme scheme_;
    double cfl_;

    // 计算残差 (通量散度)
    std::vector<State> computeResidual(
        Mesh& mesh,
        HLLC2DSolver& solver
    );

    // Forward Euler步骤
    void forwardEulerStep(
        Mesh& mesh,
        const std::vector<State>& residual,
        double dt
    );

    // RK2步骤
    void rk2Step(
        Mesh& mesh,
        HLLC2DSolver& solver,
        double dt
    );
};
```



```

// RK3 SSP步骤
void rk3SSPStep(
    Mesh& mesh,
    HLLC2DSolver& solver,
    double dt
);
};

```

6.4.3 主程序框架

```

#include <iostream>
#include <vector>
#include <cmath>
#include <string>

// 主程序
int main(int argc, char* argv[]) {
    // 1. 参数设置
    double gamma = 1.4;
    double cfl = 0.5;
    double tEnd = 1.0;
    int maxSteps = 10000;

    // 2. 读取网格
    Mesh mesh;
    if (!mesh.loadFromFile("mesh.dat")) {
        std::cerr << "Failed to load mesh!" << std::endl;
        return 1;
    }
    mesh.computeGeometry();

    // 3. 初始化流场
    initializeFlow(mesh);

    // 4. 创建求解器
    HLLC2DSolver solver(gamma);
    TimeIntegrator integrator(TimeIntegrator::RK2, cfl);

    // 5. 时间推进
    double t = 0.0;
    int step = 0;

    while (t < tEnd && step < maxSteps) {
        // 5.1 计算时间步长
        double dt = integrator.computeTimeStep(mesh);
        if (t + dt > tEnd) dt = tEnd - t;

        // 5.2 更新节点速度 (Nodal Solver)
        solver.updateNodalVelocities(
            mesh.vertices, mesh.edges, mesh.cells

```

```

    );

    // 5.3 时间推进
    integrator.step(mesh, solver, dt);

    // 5.4 更新网格 (ALE框架)
    mesh.updatePositions(dt);
    mesh.computeGeometry();

    // 5.5 输出和监控
    t += dt;
    step++;

    if (step % 100 == 0) {
        std::cout << "Step: " << step
                    << ", Time: " << t
                    << ", dt: " << dt << std::endl;
    }

    // 5.6 检查网格质量
    double quality = mesh.computeMeshQuality();
    if (quality < 0.1) {
        std::cout << "Warning: Mesh quality low, consider remeshing\n";
    }
}

// 6. 输出结果
saveResults(mesh, "result.vtk");

return 0;
}

// 初始化流场 (以Sod激波管为例)
void initializeFlow(Mesh& mesh) {
    for (auto& cell : mesh.cells) {
        double x = cell.centroid.x;

        // Sod激波管初始条件
        double rho, u, v, p;
        if (x < 0.5) {
            rho = 1.0;
            u = 0.0;
            v = 0.0;
            p = 1.0;
        } else {
            rho = 0.125;
            u = 0.0;
            v = 0.0;
            p = 0.1;
        }

        // 转换为守恒变量
        double rhoU = rho * u;
        double rhoV = rho * v;
    }
}

```

```
        double rhoE = p / (1.4 - 1.0) + 0.5 * rho * (u*u + v*v);

        cell.state = State(rho, rhoU, rhoV, rhoE);
    }
}

// 保存结果到VTK格式
void saveResults(const Mesh& mesh, const std::string& filename) {
    std::ofstream file(filename);

    // VTK文件头
    file << "# vtk DataFile Version 3.0\n";
    file << "HLLC-2D Result\n";
    file << "ASCII\n";
    file << "DATASET UNSTRUCTURED_GRID\n";

    // 节点坐标
    file << "POINTS " << mesh.vertices.size() << " double\n";
    for (const auto& v : mesh.vertices) {
        file << v.position.x << " "
            << v.position.y << " 0.0\n";
    }

    // 单元连接 (略)
    // ...

    file.close();
}
```

第7章 数值实验简述

根据原始论文，HLLC-2D求解器经过了大量数值实验验证。以下是主要的测试案例：

7.1 一维验证问题

测试案例	描述	验证目的
Sod激波管	经典的Riemann问题	激波、接触间断捕捉
Lax问题	强激波测试	强间断稳定性
123问题	近真空状态	低密度鲁棒性
Blast波	高温高压	极端条件稳定性

7.2 二维验证问题

测试案例	描述	验证目的
二维Sod问题	圆柱形/球形激波	多维波传播
激波-涡相互作用	激波与涡的碰撞	激波捕捉精度
双马赫反射	激波在斜坡上的反射	复杂波系结构
Rayleigh-Taylor不稳定性	界面不稳定性	接触波捕捉
Richtmyer-Meshkov不稳定性	激波诱导界面不稳定性	激波-界面相互作用

7.3 Lagrangian测试

在纯Lagrangian模式下，HLLC-2D求解器通过了以下测试：

- **Sedov爆炸波**: 验证球对称激波传播
- **Noh问题**: 验证强激波汇聚
- **Kidder问题**: 验证等熵压缩
- **Saltzman活塞问题**: 验证网格运动稳定性

论文主要结论

根据原始论文, HLLC-2D求解器具有以下特点:

1. 能够精确捕捉接触间断和剪切波
2. 在强激波条件下保持良好的稳定性
3. 节点求解器确保通量守恒
4. ALE框架提供灵活的网格运动策略
5. 适用于高度变形的Lagrangian计算

第8章 总结

本文档详细介绍了HLLC-2D Riemann求解器的数学理论和C++程序框架。以下是核心要点总结：

8.1 核心创新

- 放松压力相等条件**：HLLC-2D允许接触波两侧压力不同，这是与传统HLLC的根本区别
- 节点求解器 (Nodal Solver)**：通过求解局部线性方程组计算节点接触速度，确保所有交汇边的通量守恒
- 统一的ALE框架**：通过选择不同的网格运动速度，可以退化为纯Eulerian或纯Lagrangian方法

8.2 Nodal Solver的关键公式

$$\mathbf{M}\mathbf{u}_*^q = \mathbf{b}$$

$$\mathbf{M} = \sum_k L_k (\alpha_{L,k} + \alpha_{R,k}) \mathbf{N}_k \otimes \mathbf{N}_k$$

$$\mathbf{b} = \sum_k L_k (\alpha_{L,k} + \alpha_{R,k}) \nu_{*,k} \mathbf{N}_k$$

8.3 程序实现要点

- 数据结构**：清晰分离Vertex、Edge、Cell三个核心数据结构

2. **模块化设计**: NodalSolver、HLLC2DSolver、TimeIntegrator各司其职
3. **数值稳定性**: M矩阵正则化、备选速度计算
4. **ALE灵活性**: 支持多种网格运动策略

8.4 应用建议

- 对于接触波/界面捕捉要求高的应用，推荐使用Lagrangian或混合模式
- 对于大变形问题，需要配合网格重分策略
- 初始阶段使用较小的CFL数，待流场稳定后可适当增大
- 定期监控网格质量，及时触发重分

8.5 参考文献

1. Shen Z., Yan W., Yuan G. "A robust and contact resolving Riemann solver on unstructured mesh, Part I, Euler method", Journal of Computational Physics 268 (2014) 432-455.
2. Shen Z., Yan W., Yuan G. "A robust and contact resolving Riemann solver on unstructured mesh, Part II, ALE method", Journal of Computational Physics 268 (2014) 456-484.
3. Toro E.F. "Riemann Solvers and Numerical Methods for Fluid Dynamics", Springer, 2009.
4. Donea J., Huerta A. "Finite Element Methods for Flow Problems", Wiley, 2003.

文档说明

本文档旨在帮助CFD软件工程师理解HLLC-2D求解器的核心数学思想。实际实现时，需要根据具体应用需求进行优化和调整。建议结合原始论文和本框架进行开发。